

DD: FUL-02-STEP-ACTIVATION — Subscription Activation Handoff

Developer implementation guide: DD_FUL-02-STEP-ACTIVATION-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-STEP-ACTIVATION-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md The terminal step that hands the activation off to SUB-WF-ACTIVATE-01. Once activation completes, the order is done — the customer is live, billing has charged, the network is configured. Phases: ACTIVATING → ACTIVATED (success terminal) or ACTIVATION_FAILED (failure terminal).

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-ACTIVATION-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/subscriptions/{subscription_id}/wait-for-terminal-state?processInstanceId=...&timeout=60s
- POST /api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q/activate
- POST /api/subscriptions/{subscription_id}/activate
- POST /engine-rest/external-task/{taskId}/complete
- POST /engine-rest/message

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- mark-activation-failed
- notify-success
- order-mark-activation-failed
- order-notify-success
- order-trigger-activation
- put-state
- trigger-activation
- wait-activation-outcome
- wait-install-finalize

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionActivated
- SubscriptionActivatedConsumer
- SubscriptionActivationFailed
- SubscriptionActivationFailedConsumer

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.

9. Emit success/rejected events through the outbox.

10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

The terminal step that hands the activation off to SUB-WF-ACTIVATE-01. Once activation completes, the order is done — the customer is live, billing has charged, the network is configured. Phases: ACTIVATING → ACTIVATED (success terminal) or ACTIVATION_FAILED (failure terminal).

1. What this step does

The order has reached its final hurdle. Equipment is physically installed (STEP-INSTALL captured the serial numbers); the customer has paid; KYC has cleared; the subscription exists in CREATED status. Now we have to actually turn the service on — charge the installation fees and first cycle, push the customer's profile to the AAA, configure the OLT port + VLAN, finalize the subscription's status as ACTIVE.

We don't do any of that work ourselves. SUB-WF-ACTIVATE-01 owns the activation saga. Our job in this step is to hand off to it cleanly:

- 1. Trigger the activation saga** (order-trigger-activation worker). POSTs to SUB-WF-ACTIVATE-01's trigger endpoint with the subscription ID, the activation date, the operator's cycle-anchor overflow policy, and the equipment bindings STEP-INSTALL captured. The saga responds 202 Accepted with a processInstanceId and the topics where it'll publish its outcome.
- 2. Wait for the activation to finish, via Kafka.** Camunda parks at wait-activation-outcome — a message catch event listening for either ActivationSucceeded or ActivationFailed. Two of our Kafka consumers (SubscriptionActivatedConsumer for the success topic, SubscriptionActivationFailedConsumer for the failure topic) correlate the right outcome back to the parked order.
- 3. Finalize the order.** Success → order-notify-success worker marks the order ACTIVATED, emits OrderActivated Kafka event. Failure → order-mark-activation-failed marks ACTIVATION_FAILED, emits OrderActivationFailed + OrderRefundRequested (refund cascade if money was taken).

Async handoff via 202 + Kafka outcome

SUB-WF-ACTIVATE-01 responds 202 because the activation legitimately passes through PENDING_ACTIVATION as a committed intermediate state — the row is visible in subscription table during the saga, before BIL-01 and FUL-03 have finished. Pretending the call was synchronous would be misleading. So FUL-02 accepts the 202 and waits on Kafka.

There is a long-poll convenience endpoint (GET /api/subscriptions/{subscription_id}/wait-for-terminal-state?processInstanceId=...&timeout=60s) but the SUB-WF-ACTIVATE-01 DD discourages its use ("blocks a

server-side connection per pending activation"). We use Kafka instead — same outcome, no held connections.

Idempotency via deterministic correlationId

Because the trigger is async and our worker might be re-fetched after a Camunda lock expiry (e.g., worker crash between POST and /complete), we need to be sure that re-triggering doesn't kick off two activation sagas. SUB-WF-ACTIVATE-01 accepts a correlationId in the request body for exactly this reason — if it sees the same correlationId twice for the same subscription, it returns the existing processInstanceId rather than starting a fresh saga.

We use a deterministic correlationId: act_{orderId} — derived from orderId, the same on every re-trigger of the same order. The worker computes it, posts it, accepts the 202 whether the saga is fresh or pre-existing.

Equipment binding translation

STEP-INSTALL captures equipment as an array shape: [{equipmentRequirement, equipmentRef, serialNumber}]. SUB-WF-ACTIVATE-01 wants a named-slot map: {ontSerial: "...", stbSerials: [...], ...}. The worker translates between the two using a static mapping per equipmentRequirement → SUB-WF-ACTIVATE-01 slot name (e.g., GPON_ONT → ontSerial; ATA → ataSerial; STB → stbSerials[]). The mapping table lives in ful_02_config (one row per operator, with equipment_slot_mapping_jsonb per FRAMEWORK \$5.3 future extension; v1.0 ships with a hardcoded default).

Refund cascade on activation failure

If activation fails after the customer has paid, the customer is owed a refund — same pattern as STEP-INSTALL's failure path. order-mark-activation-failed emits OrderRefundRequested alongside OrderActivationFailed when payment_received_amount > 0. BIL-02-ADJ-01 queues the refund.

What this step deliberately doesn't do

- **Doesn't charge billing.** BIL-01 charging happens inside SUB-WF-ACTIVATE-01 — that workflow's responsibility (per its R-WF-2). FUL-02 just triggers; the saga handles the charging.
- **Doesn't provision the network.** FUL-03 owns network provisioning. SUB-WF-ACTIVATE-01 calls FUL-03 synchronously from its own service task; FUL-02 doesn't see FUL-03 in this step.
- **Doesn't roll back the subscription on failure.** SUB-WF-ACTIVATE-01 owns its own rollback logic — on failure, it reverts the subscription to CREATED and writes last_failure onto the subscription row. FUL-02 just marks ACTIVATION_FAILED on its own customer_order row; the subscription stays alive in CREATED for ops to investigate (or terminate via SUB-WF-TERMINATE-01 if the activation can't be retried).

2. Worked example end-to-end (WIK, ACTIVATED)

Continuing John Mwangi's order from STEP-INSTALL. Order ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1, subscription sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q, equipment captured at 15:35: ONT serial HW_ONT_NK4501, ATA serial HW_ATA_AT8901. Camunda just advanced past wait-install-finalize to trigger-activation.

Step 1 — Worker translates equipment bindings and triggers the saga

TriggerActivationWorker picks up the external task. Process variables: orderId, subscriptionId, equipmentBindings (Json from STEP-INSTALL), operatorCode, optional intendedActivationDate.

Worker reads operator activation config from ful_02_config:

```
SELECT cycle_anchor_overflow_policy
FROM ful_02_config
WHERE operator_code = 'WIK';
```

→ cycle_anchor_overflow_policy = 'ROUND_TO_28' (WIK chose to round day-29/30/31 anchors to day 28).

Worker translates the equipment-bindings array into named slots:

```
Input (from STEP-INSTALL):
[
  {"equipmentRequirement": "GPON_ONT", "equipmentRef": "ont_HW_ONT_NK4501", "serialNumber": "HW_ONT_NK4501"},
  {"equipmentRequirement": "ATA", "equipmentRef": "ata_HW_ATA_AT8901", "serialNumber": "HW_ATA_AT8901"}
]

Translated (for SUB-WF-ACTIVATE-01):
{
  "ontSerial": "HW_ONT_NK4501",
```

```

    "ataSerial": "HW_ATA_AT8901"
}

```

Worker computes deterministic correlationId: act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1.

POSTs to SUB-WF-ACTIVATE-01:

```

POST /api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q/activate HTTP/1.1
Host: subscription-workflow-activate.sophix.internal
Authorization: Bearer <module-svc-creds, role SUBSCRIPTION_OPERATOR>
Content-Type: application/json

```

```

{
  "correlationId": "act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "intendedActivationDate": "2026-05-18",
  "backdated": false,
  "cycleAnchorOverflowPolicy": "ROUND_TO_28",
  "equipmentBindings": {
    "ontSerial": "HW_ONT_NK4501",
    "ataSerial": "HW_ATA_AT8901"
  },
  "channel": "FULFILLMENT_ORDER_CAPTURE",
  "actor": "fulfillment-order-capture-worker-9"
}

```

SUB-WF-ACTIVATE-01 responds 202 Accepted:

```

{
  "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
  "processInstanceId": "proc_01HZX_ACTIVATE_42",
  "pendingStatus": "PENDING_ACTIVATION",
  "links": {
    "self": "/api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "waitForTerminalState": "/api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q/wait-for-terminal-state?processInstanceId=proc_01HZX_ACTIVATE_42",
    "kafkaTopics": [
      "sophix.subscription.subscription.activated",
      "sophix.subscription.subscription.activationfailed"
    ]
  }
}

```

Worker UPDATES:

```

UPDATE customer_order
  SET activation_correlation_id = 'act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
      activation_process_instance_id = 'proc_01HZX_ACTIVATE_42',
      last_updated_at = NOW()
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';

```

Emits sophix.orders.activating:

```

{
  "eventType": "OrderActivating",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T15:35:30.123Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "activationCorrelationId": "act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "activationProcessInstanceId": "proc_01HZX_ACTIVATE_42",
    "triggeredAt": "2026-05-18T15:35:30Z"
  }
}

```

Worker reports completion to Camunda:

```

POST /engine-rest/external-task/{taskId}/complete HTTP/1.1

{
  "workerId": "fulfillment-order-capture-worker-9",
  "variables": {
    "activationCorrelationId": { "value": "act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
    "activationProcessInstanceId": { "value": "proc_01HZX_ACTIVATE_42", "type": "String" }
  }
}

```

Camunda 204. BPMN advances to wait-activation-outcome (message catch, no boundary timer in v1.0 — see R-FUL-02-ACT-7).

PhaseProjector updates:

```

UPDATE customer_order

```

```

SET current_phase          = 'ACTIVATING',
   current_activity_id     = 'wait-activation-outcome',
   last_updated_at        = NOW()
WHERE order_id            = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';

```

Step 2 — SUB-WF-ACTIVATE-01 runs its saga (out-of-FUL-02 visibility)

The saga does (per its own DD):

1. Snapshot the Package + cycle config.
2. Resolve the cycle anchor day (today = 18 May, day-of-month = 18, within 1-28 range → no overflow policy needed; cycleAnchorDay = 18).
3. Call BIL-01 with ACTIVATION_INTENT to charge installation fee + first cycle prorate. (BIL-01 deducts from John's credit balance of 15000 KES which was loaded at STEP-PAYMENT.)
4. Call FUL-03 to push the customer profile to AAA, provision the GPON port for the ONT, configure the ATA for VoIP.
5. SUB-LM-01 put-state to ACTIVE, write cycle_anchor_day = 18, activated_at = 2026-05-18T15:36:42Z, equipment_bindings = {ontSerial: HW_ONT_NK4501, ataSerial: HW_ATA_AT8901}.

After ~52 seconds total, the saga commits. SUB-WF-ACTIVATE-01 emits sophix.subscription.subscription.activated:

```

{
  "eventType": "SubscriptionActivated",
  "aggregateId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
  "timestamp": "2026-05-18T15:36:42.567Z",
  "payload": {
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "packageRef": "pkg_INET_VOIP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "processInstanceId": "proc_01HZX_ACTIVATE_42",
    "correlationId": "act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "cycleAnchorDay": 18,
    "cycleAnchorOverflowApplied": "NONE",
    "activatedAt": "2026-05-18T15:36:42Z",
    "totalSagaDurationSeconds": 52
  }
}

```

Step 3 — Our consumer wakes Camunda

SubscriptionActivatedConsumer receives the event:

1. Look up the order by activation_correlation_id = payload.correlationId:

```

SELECT order_id, current_phase
FROM customer_order
WHERE activation_correlation_id = 'act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';

```

→ matches ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1, phase ACTIVATING.

(Fallback if correlation ID is missing — shouldn't happen but defensive: look up by subscription_id. Documented in R-FUL-02-ACT-3.)

2. Idempotency check: customer_order_processed_activations(order_id, process_instance_id) UNIQUE.
3. Wake Camunda:

```

POST /engine-rest/message HTTP/1.1
{
  "messageName": "ActivationSucceeded",
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processVariables": {
    "activatedAt": { "value": "2026-05-18T15:36:42Z", "type": "String" },
    "cycleAnchorDay": { "value": 18, "type": "Integer" },
    "cycleAnchorOverflowApplied": { "value": "NONE", "type": "String" },
    "totalSagaDurationSeconds": { "value": 52, "type": "Integer" }
  }
}

```

Camunda wakes wait-activation-outcome and advances to the success-side worker order-notify-success.

Step 4 — order-notify-success finalizes the order

```
UPDATE customer_order
SET current_phase = 'ACTIVATED',
    activated_at = '2026-05-18 15:36:42',
    last_updated_at = NOW()
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Worker emits `sophix.orders.activated`:

```
{
  "eventType": "OrderActivated",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T15:36:43.123Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "packageRef": "pkg_INET_VOIP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "activatedAt": "2026-05-18T15:36:42Z",
    "totalElapsedSeconds": 23802,
    "salesChannel": "FIELD_AGENT_PWA",
    "activatedBy": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1"
  }
}
```

`totalElapsedSeconds` = 23802 = 6h 36min — from STEP-CAPTURE 09:00 EAT (06:00 UTC) to ACTIVATED 15:36 UTC. WIK's target is <8 hours for in-coverage same-day activations; this one made it.

Worker completes the Camunda task. BPMN ends. Order is done.

DD_NOT-01 consumes `OrderActivated` and SMSes John: "Welcome to Zuku Fiber! Your 100M internet is now live. Try it: open a browser and visit zuku.co.ke."

Total elapsed in step: ~67 seconds (the saga's 52 seconds + ~15 seconds of FUL-02 overhead).

3. Worked example — activation failure

Variant: at step 2, BIL-01 returns an error during charging (customer's credit balance was somehow drained between STEP-PAYMENT and now — perhaps an edge-case ops adjustment).

SUB-WF-ACTIVATE-01 reverts and emits failure

The saga reverts: SUB-LM-01 puts state back to `CREATED`, writes `last_failure` JSONB, emits `sophix.subscription.subscription.activationfailed`:

```
{
  "eventType": "SubscriptionActivationFailed",
  "aggregateId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
  "timestamp": "2026-05-18T15:36:18.234Z",
  "payload": {
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "processInstanceId": "proc_01HZX_ACTIVATE_42",
    "correlationId": "act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "failure": {
      "operation": "ACTIVATION",
      "category": "BILLING",
      "reasonCode": "INSUFFICIENT_CREDIT_BALANCE",
      "reasonDetail": "Customer credit balance 0 KES < required 15000 KES for INSTALLATION_FEE + FIRST_CYCLE_PRORATE",
      "actionable": true,
      "processInstanceId": "proc_01HZX_ACTIVATE_42"
    },
    "revertedToStatus": "CREATED",
    "failedAt": "2026-05-18T15:36:18Z"
  }
}
```

Our consumer wakes Camunda on the failure side

`SubscriptionActivationFailedConsumer`:

1. Look up by `correlationId` → `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`.
2. Idempotency check.

3. Wake Camunda:

```
POST /engine-rest/message HTTP/1.1
{
  "messageName": "ActivationFailed",
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processVariables": {
    "activationFailureCategory": { "value": "BILLING", "type": "String" },
    "activationFailureCode": { "value": "INSUFFICIENT_CREDIT_BALANCE", "type": "String" },
    "activationFailureDetail": { "value": "Customer credit balance 0 KES < required 15000 KES...", "type": "String" }
  }
}
```

Camunda wakes wait-activation-outcome, routes to order-mark-activation-failed.

order-mark-activation-failed runs

```
UPDATE customer_order
SET current_phase = 'ACTIVATION_FAILED',
    failure_reason = 'ACTIVATION_FAILED_BILLING',
    failure_detail = 'INSUFFICIENT_CREDIT_BALANCE: Customer credit balance 0 KES < required 15000 KES for INSTALLATION_FEE + FIRST_CYCLE_PRORATE',
    last_updated_at = NOW()
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Worker reads payment_received_amount — 15000.00 KES. Customer paid; this is a refund candidate (the subscription is back in CREATED, billing was reverted, no actual charge has settled — but the customer's money is in the credit balance).

Worker emits OrderRefundRequested then OrderActivationFailed:

```
{
  "eventType": "OrderActivationFailed",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T15:36:19.123Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "subscriptionStatus": "CREATED",
    "failureCategory": "BILLING",
    "failureCode": "INSUFFICIENT_CREDIT_BALANCE",
    "failureDetail": "Customer credit balance 0 KES < required 15000 KES for INSTALLATION_FEE + FIRST_CYCLE_PRORATE",
    "paymentReceivedAmount": 15000.00,
    "currency": "KES",
    "refundRequested": true,
    "failedAt": "2026-05-18T15:36:19Z"
  }
}
```

subscriptionStatus = CREATED tells consumers the subscription is alive but not active — ops may decide whether to retry activation (after fixing the credit balance issue), terminate the subscription, or escalate. FUL-02 doesn't auto-retry — the order is terminal at this point.

DD_NOT-01 → John: "Sorry, we couldn't complete your activation due to a billing issue. Please contact support; you'll receive a refund within 5 business days."

4. Workers

4.1 order-trigger-activation

Field	Value
Topic	order-trigger-activation
Purpose	Translate equipment bindings to SUB-WF-ACTIVATE-01's named-slot shape; compute deterministic correlationId; POST activate trigger. Accept the 202 and let Camunda wait on Kafka for the outcome.
Process variables read	orderId, subscriptionId, equipmentBindings (from STEP-INSTALL), operatorCode, optional intendedActivationDate
Config read	ful_02_config.cycle_anchor_overflow_policy (e.g., ROUND_TO_28 for WIK). ful_02_config.equipment_slot_mapping_jsonb for the equipmentRequirement → SUB-WF-ACTIVATE-01-slot translation.
External HTTP calls	SUB-WF-ACTIVATE-01 POST /api/subscriptions/{subscription_id}/activate
Process variables written	activationCorrelationId, activationProcessInstanceId

Field	Value
customer_order columns written	activation_correlation_id, activation_process_instance_id
Kafka emissions	sophix.orders.activating
Lock duration	95 000 ms (saga itself takes up to 90 sec per SUB-WF-ACTIVATE-01 R-WF-3; we add 5 sec margin to keep the worker alive through the POST returning 202)
Retry policy	0 retries on /failure. The trigger is idempotent (deterministic correlationId); a re-fetch after lock expiry will safely re-POST and receive the existing processInstanceId.

Failure modes:

- **SUB-WF-ACTIVATE-01 returns 4xx** (validation failure — e.g., subscription already ACTIVE, equipment slot not recognized) → /failure with the error → BPMN routes to order-mark-activation-failed.
- **SUB-WF-ACTIVATE-01 unreachable / 5xx** → /failure with UPSTREAM_UNREACHABLE. Incident.

4.2 order-notify-success

Field	Value
Topic	order-notify-success
Purpose	Terminal success handler. Marks the order ACTIVATED, emits OrderActivated Kafka event.
Process variables read	orderId, customerId, subscriptionId, packageRef, homepassId, operatorCode, salesChannel, activatedAt, actor, capturedAt (from earlier in the BPMN)
External HTTP calls	None
Process variables written	None (terminal)
customer_order columns written	current_phase = ACTIVATED, activated_at
Kafka emissions	sophix.orders.activated
Lock duration	5 000 ms
Retry policy	3 retries with PT30S

4.3 order-mark-activation-failed

Field	Value
Topic	order-mark-activation-failed
Purpose	Terminal failure handler for activation path. Marks ACTIVATION_FAILED, emits failure + refund events.
Process variables read	orderId, subscriptionId, activationFailureCategory, activationFailureCode, activationFailureDetail, operatorCode
External HTTP calls	None
Process variables written	None (terminal)
customer_order columns written	current_phase = ACTIVATION_FAILED, failure_reason, failure_detail
Kafka emissions	sophix.orders.activation-failed. If payment_received_amount > 0: also sophix.orders.refund-requested.
Lock duration	5 000 ms
Retry policy	3 retries with PT30S

5. Message catch events

5.1 ActivationSucceeded

Triggered by: SubscriptionActivatedConsumer after receiving sophix.subscription.subscription.activated.

Where BPMNs park: <bpmn:intermediateCatchEvent> named wait-activation-outcome.

Boundary timer: NONE in v1.0 (see R-FUL-02-ACT-7). SUB-WF-ACTIVATE-01 has its own 90-second timeout; if it never emits an event, that's an ops escalation, not auto-failure of the order.

Process variables set on correlation: `activatedAt`, `cycleAnchorDay`, `cycleAnchorOverflowApplied`, `totalSagaDurationSeconds`.

5.2 ActivationFailed

Triggered by: `SubscriptionActivationFailedConsumer` after receiving `sophix.subscription.subscription.activationfailed`.

Where BPMNs park: same wait-activation-outcome (BPMN has two outgoing message branches).

Process variables set on correlation: `activationFailureCategory`, `activationFailureCode`, `activationFailureDetail`.

5.3 SubscriptionActivatedConsumer + SubscriptionActivationFailedConsumer

Field	SubscriptionActivatedConsumer	SubscriptionActivationFailedConsumer
Subscribes to	<code>sophix.subscription.subscription.activated</code>	<code>sophix.subscription.subscription.activationfailed</code>
Lookup	By <code>activation_correlation_id = payload.correlationId</code> on <code>customer_order</code> . Fallback: by <code>subscription_id</code> .	Same
Idempotency	<code>customer_order_processed_activations(order_id, process_instance_id) UNIQUE</code>	Same
customer_order columns written	None — write happens in order-notify-success worker (after Camunda confirms the message is correlated successfully)	None
Camunda call	POST <code>/engine-rest/message ActivationSucceeded</code>	POST <code>/engine-rest/message ActivationFailed</code>
DLQ	On no match: log DEBUG, skip. SUB-WF-ACTIVATE-01 may be activating subscriptions not originating from FUL-02 (admin direct activations); we ignore those.	Same

6. DB tables touched

customer_order (FRAMEWORK §5.1) — UPDATE

Column	When set	Set by	Example value
<code>activation_correlation_id</code>	When trigger POST succeeds	<code>order-trigger-activation</code>	<code>act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1</code>
<code>activation_process_instance_id</code>	When trigger POST succeeds	<code>order-trigger-activation</code>	<code>proc_01HZX_ACTIVATE_42</code>
<code>activated_at</code>	On success only	<code>order-notify-success</code>	<code>2026-05-18 15:36:42</code>
<code>current_phase</code>	On BPMN transitions	<code>PhaseProjector</code>	<code>READY_FOR_INSTALL/INSTALLING → ACTIVATING → ACTIVATED</code> or <code>ACTIVATION_FAILED</code>
<code>current_activity_id</code>	On each transition	<code>PhaseProjector</code>	<code>trigger-activation → wait-activation-outcome → notify-success (success) or mark-activation-failed (failure)</code>
<code>failure_reason</code> (on failure)	<code>order-mark-activation-failed</code>	<code>ACTIVATION_FAILED_BILLING / ACTIVATION_FAILED_FULFILLMENT / ACTIVATION_FAILED_TIMEOUT</code>	
<code>failure_detail</code> (on failure)	<code>order-mark-activation-failed</code>	Free-text from SUB-WF-ACTIVATE-01's <code>failure.reasonDetail</code>	
<code>last_updated_at</code>	On every UPDATE	<code>trigger / consumer / worker</code>	<code>2026-05-18 15:36:42.789</code>

Sample data — five orders at various activation stages:

order_id	operator_code	activation_correlation_id	activated_at	current_phase	failure_reason
ord_01HZ...A	WIK	act_ord_01HZ...A	2026-05-18 15:36:42	ACTIVATED	NULL
ord_01HZ...B	WIK	act_ord_01HZ...B	NULL	ACTIVATING (saga in flight)	NULL
ord_01HZ...C	WIK	act_ord_01HZ...C	NULL	ACTIVATION_FAILED (billing — refund pending)	ACTIVATION_FAILED_BILLING
ord_01HZ...D	WIK	act_ord_01HZ...D	NULL	ACTIVATION_FAILED (FUL-03 provisioning)	ACTIVATION_FAILED_FULFILLMENT
ord_01HZ...E	WIK	act_ord_01HZ...E	2026-05-19 14:32:18	ACTIVATED	NULL

customer_order_processed_activations — NEW IDEMPOTENCY TABLE

Column	Type	Notes
id	BIGSERIAL PK	
order_id	TEXT NOT NULL	FK to customer_order
process_instance_id	TEXT NOT NULL	From SUB-WF-ACTIVATE-01's event payload
outcome	TEXT NOT NULL	Values: ACTIVATED / FAILED
processed_at	TIMESTAMP NOT NULL	When our consumer processed it

Indexes: PK; UNIQUE (order_id, process_instance_id). P5Y retention.

Sample data:

order_id	process_instance_id	outcome	processed_at
ord_01HZ...A	proc_01HZX_ACTIVATE_42	ACTIVATED	2026-05-18 15:36:42.789
ord_01HZ...C	proc_01HZX_ACTIVATE_50	FAILED	2026-05-18 15:36:19.412

7. Kafka events emitted

sophix.orders.activating

```
{
  "eventType": "OrderActivating",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T15:35:30.123Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "activationCorrelationId": "act_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "activationProcessInstanceId": "proc_01HZX_ACTIVATE_42",
    "triggeredAt": "2026-05-18T15:35:30Z"
  }
}
```

sophix.orders.activated

```
{
  "eventType": "OrderActivated",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T15:36:43.123Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "packageRef": "pkg_INET_V0IP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "activatedAt": "2026-05-18T15:36:42Z",
    "totalElapsedSeconds": 23802,
    "salesChannel": "FIELD_AGENT_PWA",
    "activatedBy": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1"
  }
}
```

```
}
```

sophix.orders.activation-failed

```
{
  "eventType": "OrderActivationFailed",
  "aggregateId": "ord_01HZ...C",
  "timestamp": "2026-05-18T15:36:19.123Z",
  "payload": {
    "orderId": "ord_01HZ...C",
    "customerId": "cust_01HX...C",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX...C",
    "subscriptionStatus": "CREATED",
    "failureCategory": "BILLING",
    "failureCode": "INSUFFICIENT_CREDIT_BALANCE",
    "failureDetail": "Customer credit balance 0 KES < required 15000 KES for INSTALLATION_FEE + FIRST_CYCLE_PRORATE",
    "paymentReceivedAmount": 15000.00,
    "currency": "KES",
    "refundRequested": true,
    "failedAt": "2026-05-18T15:36:19Z"
  }
}
```

Consumers:

Consumer	Purpose
DD_NOT-01	On OrderActivated: customer welcome SMS+email with instructions for verifying the service. On OrderActivationFailed: SMS with refund timeline.
CVM	Funnel terminal stage — OrderActivated is the conversion event for all customer-acquisition reporting. Activation-failure rate dashboards split by failureCategory.
Finance	OrderActivated carries salesChannel + totalElapsedSeconds for agent commission + SLA reporting.
BIL-02-ADJ-01	On OrderRefundRequested (activation-failure variant): refund queue.

8. Step-pinned rules

ID	Rule
R-FUL-02-ACT-1	order-trigger-activation POSTs to SUB-WF-ACTIVATE-01 POST /api/subscriptions/{subscription_id}/activate with deterministic correlationId = act_{orderId}. Re-trigger of the same order receives the existing processInstanceId rather than starting a fresh saga.
R-FUL-02-ACT-2	Equipment bindings are translated from STEP-INSTALL's array shape ([{equipmentRequirement, equipmentRef, serialNumber}]) to SUB-WF-ACTIVATE-01's named-slot map ({ontSerial, ataSerial, stbSerials, ...}) using ful_02_config.equipment_slot_mapping_jsonb. v1.0 ships the WIK default mapping (GPON_ONT → ontSerial, ATA → ataSerial, STB → stbSerials[]).
R-FUL-02-ACT-3	SubscriptionActivatedConsumer and SubscriptionActivationFailedConsumer correlate primarily via activation_correlation_id. Fallback: subscription_id. On neither matching, log DEBUG and skip — the activation is for a subscription not originating in FUL-02 (e.g., admin direct activation).
R-FUL-02-ACT-4	Trigger worker uses lock duration 95 sec (saga can take up to 90 sec per SUB-WF-ACTIVATE-01 R-WF-3; +5 sec margin for the worker to receive the 202 and /complete).
R-FUL-02-ACT-5	Activation failure with payment_received_amount > 0 triggers refund cascade — order-mark-activation-failed emits both OrderActivationFailed and OrderRefundRequested. The subscription remains in CREATED status (SUB-WF-ACTIVATE-01 reverted it); FUL-02 does NOT terminate the subscription from this step. Ops decides whether to retry activation or to terminate the subscription via SUB-WF-TERMINATE-01.
R-FUL-02-ACT-6	FUL-02 does NOT roll back BIL-01 charges or FUL-03 provisioning on activation failure — SUB-WF-ACTIVATE-01 owns its own rollback (per its R-A-5 / R-WF-* family). FUL-02 just reflects the outcome in its own customer_order row.

ID	Rule
R-FUL-02-ACT-7	No boundary timer on wait-activation-outcome in v1.0. SUB-WF-ACTIVATE-01 has its own 90-second timeout that emits SubscriptionActivationFailed with reasonCode = BILLING_TIMEOUT or FULFILLMENT_TIMEOUT; FUL-02 receives that via Kafka. A separate FUL-02-side timer would race the saga's own timeout and create double-reporting. If the saga never emits any event (catastrophic — Camunda crash inside the saga), the order is parked indefinitely; ops sees it via NOC dashboards driven by OrderActivating timestamps and escalates.
R-FUL-02-ACT-8	At-least-once Kafka delivery: customer_order_processed_activations(order_id, process_instance_id) UNIQUE. Re-processing of the same SubscriptionActivated/SubscriptionActivationFailed event is skipped silently.